

CS452-a0-Description

Program Structure

- The structure of my program is as follows: I have a polling loop in which I would have a queue that contains all the instructions for the Marklin controller. The queue is to be popped every 1.5 tenths of a second to not overload the Marklin controller.
- I have two variables I used to read the Marklin: `polling_s88` and `polling_s88_byte`. `polling_s88` defines the s88 that the program is interested in reading and `polling_s88_byte` is the byte number. When I call `read_multiple_s88` or `0x85` it alerts the Marklin to start sending information on it's sensors back to the raspberry pi. Since the pi is too fast to handle concurrently the commands are to be put into the `FR` queue. I must keep a counter to keep track of the bytes sent back as their order is important. I need to know which byte corresponding to which sensor on the track enable to display the recently pressed sensors.
- The raspberry pi would only do another read if it is not in read already and there are no commands in the execution queue. The conditions are `polling_s88 = 0` and `UART_REG(line, UART_FR) & UART_FR_RXFE == 0`. `polling_s88` is set to 0 right after the program finishes reading from the response queue. `UART_REG(line, UART_FR) & UART_FR_RXFE == 0` when there are no bytes in the input for the pi. Since the marklin sends the commands slower than the pi can process, one cannot expect to get the full input every loop without running the risk of a polling loop being longer than 0.1 seconds. If there are commands in the execution queue, then there may be a backup and my "circular queue" structure would fall apart as it would overflow. Some commands would not be able to be executed as they are overridden by the pointers.
 - One drawback to this design is the Marklin would not be able to read from the tracks when it is in the middle of executing commands "i.e reversing a train". However even though it is able to poll the track the information is useless as it cannot be react upon.
- I also employed what I call "lazy print" in which I would print the elements atomically. That is instead of refreshing the entire screen, I would only reprint the section that have changed. For example when a switch changed. I would just reprint that single switch instead of refreshing the screen. Such eliminates the print bottleneck and drastically decrease my polling loop time. The same for the live switches and the recently activated the bytes are updated on read individually.
- I also modified the `get_c` command in such that it would not buzywait for the next character. If there exist no character in the queue the modified command would just return a 0 character and move on. One cannot busywait as there are other tasks needed to be done when there exist no character input, i.e checking for track readings.

- I use a char array as the string for my command. One can type command and even backspace mistakes. I also have a parse command function that would parse the command into the parameters and then enqueue the marklin command.
- When the program is ran it goes through a track initialization phase in which it stops the trains, sets all track direction to straight and then to curved. Such is meant to be a visual and auditory hardware debugger to see which solenoid have been burnt. I have incorporated my lazy print strategy with my switch update. Thus the screen updates exactly the same time as the track update. There are no prior update on the screen then track. I also have a marklin read test at program start to verify if it is possible to connect to the marklin. If either the marklin is off or disconnected one must reboot the pi as I utilized the old `get_c` busywait function. This in turn gets the read time of the marklin.

We will judge your program primarily on the basis of this description. Describe which algorithms and data structures are used and why you chose them.

- the data structures I implemented in the assignment are strings, circular queues and circular lists.
- "circular queue" is a list with two pointers, the head pointer and the end pointer. Every time I enqueue an element I would set the element pointed by the end pointer to the enqueued element and increment the end pointer. When I pop the queue I would just increment the head pointer. If the end or head pointer reaches the end of the list it would just start from the front again. A queue structure is needed as the marklin is unable to process commands sequentially unlike the pi. Thus a queue with an execution delay is needed to communicate with the Marklin. The circular queue is also a two dimensional array as most marklin commands are two bytes.
 - The circular queue is easy to implement as compared to a linked list. For a linked list I need to allocate memory and judging there is no free function. I am aware I am not going to get allocated memory back after elements are popped from the list. Thus a circular queue provides a convenient way to implement queue like functionality without much complexity. One drawback is the queue only have limited capacity. However the queue I set can hold 200 commands and read commands are only enqueued when the queue is empty and when there is not a read in process.
- I also maintained a simply "circular list" to keep track of the recent sensor inputs. A circular list is much like a list however with only one pointer maintained. The pointer to the back. Since I am only going to print the 10 most recent entries I do not need to keep a large dynamically sized list. I can just iterate backwards from the last element. I can easily get the 10 most recent sensor commands to be printed on the screen.

This description should include a list of unimplemented aspects of the assignments, if any. Also, if you know of bugs in your implementation describe them explicitly.

Answers to the following questions:

1. How do you know that your clock does not miss updates or lose time?
 1. I ensured every polling loop is under 100ms. This way the clock would not skip time. I would also use the system-timer call since the system timer is consistent. Ensure the time of EVERY polling loop is less than 100 ms including reads.
2. How long does the train hardware take to reply to a sensor query? (Note: To answer these questions, you need to perform and report some timing of your polling loop.)
 1. An iteration of my polling loop is around $3991\ \mu s$
 2. A hardware sensor query reply is: $71412\ \mu s$
 3. Such data is already in the program for one to see.

Features

1. Train Control

- **Train Movement:** The program allows users to control the speed and direction of individual model trains. Users can issue commands such as `tr <train number> <train speed 1 - 14>` to control the speed of a specific train and `rv <train number>` to reverse the direction of a train.
- To set attributes to the train use `tr <train number> <attribute>`

```
code functions on
64 all off
65 #1
66 #2
67 #1,#2
68 #3
69 #3,#1
70 #3,#2
71 #3,#2,#1
72 #4
73 #4,#1
74 #4,#2
75 #4,#2,#1
76 #4,#3
77 #4,#3,#1
78 #4,#3,#2
79 #4,#3,#2,#1
```

2. Switch Control

- **Switch Management:** Users can control switches on the model train layout. Commands like `sw <switch number> <switch direction 'C' or 'S'>` enable users to change the position of switches, facilitating train routing.

3. Data Structures

- **Marklin Command Queue:** To ensure efficient command processing, the program employs a command queue. This queue prevents overloading the Marklin controller, which has a processing speed limitation of 100ms per command. I used a list as a queue that would send data to the marklin controller .
- ******Print Activated Sensors List:****** This list contains all the recently activated sensors. It also employs the circular method and two pointers. Whenever a new sensor is added into the list the second pointer moves one space to the right.

4. Solenoid Protection

- **Solenoid Safety:** The program includes solenoid protection logic to prevent conflicting switch configurations. It ensures that solenoids are set appropriately based on switch states. For every track switch command issued I also appended the solenoid off command immediately after the turn on command.

Usage

The program accepts commands via a command-line interface (CLI). Users can input commands to control trains, switches, and solenoids as described in the "Features" section.

To run the program go into the directory `cs452-trains/a0` .

Execute the command `make` . An image file would have been made if the build is successful. The image file would be named `d273liu.img`

Then pull the image from the workstation computer:

1. `dhcp`
2. `tftpboot 0x200000 129.97.167.60:images/user.img`, where "user" is the userid you used to upload your image file
3. `go 0x200000`

Requirements

- Compatible model train layout with a Marklin controller.
- Appropriate hardware interface for communication with the layout components.

Prerequisites

Before running the Model Train Control Program, please ensure the following prerequisites are met:

1. **Marklin System:** The Marklin train control system must be up and running. Ensure that the Marklin hardware, including the tracks, switches, and trains, is set up correctly.
2. **Marklin Reset:** It is essential to reset the Marklin system to its initial state. This ensures that the program can establish communication and control the trains effectively.
3. **Reboot Raspberry Pi:** Reboot or power on your Raspberry Pi device and ensure it is connected to the workstation.

Startup Procedure

Follow these steps to start the Model Train Control Program:

1. **Marklin System:** Make sure your Marklin system is powered on and functioning correctly. Check that all components are connected and operational. Preferably press go and stop at once to reset the Marklin.
2. **Reboot Raspberry Pi:** Reboot or power on your Raspberry Pi device and ensure it is connected to the workstation.
3. **Build the Program:** Run `make` on in the directory `cs451-trains/a0` to start building an image file. The final image file would be `d273liu.img`
4. Upload the image to <https://cs452.student.cs.uwaterloo.ca/dashboard/>. Click choose file and choose the `d273liu.img` file. Click upload
5. Run the command on the linux U-boot `tftpboot 0x200000 129.97.167.60:images/user.img`, where "user" is the userid you used to upload your image file
6. **Program Execution:** Execute the Model Train Control Program on the Raspberry Pi by running `go 0x200000`. The program will establish communication with the Marklin system and an automatic track initialization would commence.

By following these steps, you can ensure that the Model Train Control Program operates seamlessly with your Marklin setup. It is essential to have the Marklin system in a known state to achieve accurate and reliable train control.

Automatic Track Initialization

Once the Model Train Control Program is initiated on the Raspberry Pi, it operates in an autonomous mode where it continuously processes and executes commands without accepting manual inputs from the Pi. This is designed to maintain the integrity and consistency of train operations. In this mode the tracks are initially set to straight then to curved. Such can be used as a hardware debugging mechanism to see and hear which solenoids are burnt.

During program execution, please refrain from manual input or adjustments directly through the Raspberry Pi interface, as these inputs may not be processed until the program completes its current task or enters a receptive state.

To stop the program or make any adjustments, just press `q` to stop for program termination or modification, ensuring a safe and controlled transition.

On the bottom of the GUI, there exist

"Model Train Control Program 1f438fc35677473f8e243df518035009/Untitled.png" is not created yet. Click to create.

The LIVE SENSOR interface displays the real time information of the sensors on the track. The i th bit is the i th sensor.

"Model Train Control Program 1f438fc35677473f8e243df518035009/Untitled 1.png" is not created yet. Click to create.

The ACTIVATED SENSOR interface displays which sensor recently underwent a rising edge. This suggest a train have passed over the sensor

"Model Train Control Program 1f438fc35677473f8e243df518035009/Untitled 2.png" is not created yet. Click to create.

The recently activated sensors interface displays a list in real time. A new sensor is added to the top when it is triggered.

"Model Train Control Program 1f438fc35677473f8e243df518035009/Untitled 3.png" is not created yet. Click to create.

The SW column displayes the rail switches. S if it is straight and C if it is curved.

Queue